

Pre-requisites for Lab 3.2: Setting Up MCP Log Reader with MCPO Proxy

A Step-by-Step Walkthrough for OpenWebUI Integration

Overview

This walkthrough guides you through setting up a complete log analysis pipeline using two independent tools:

- **log-reader-mcp** — an MCP server that reads log files from disk
- **mcpo** — a proxy that wraps the MCP server and exposes it as a standard HTTP/OpenAPI endpoint that OpenWebUI can use

Once set up, you will be able to toggle the MCPO Proxy tool inside OpenWebUI and ask an AI model to read, filter, and analyze your system logs in natural language.

How It Works

The components interact in the following order:

```
logs/logs.log
  |
  v
log-reader-mcp  (MCP server — reads log file via stdio)
  |
  v
mcpo            (HTTP proxy — wraps MCP server, exposes OpenAPI on port 8000)
  |
  v
OpenWebUI      (connects to mcpo, gives LLM access to the tool)
  |
  v
LLM (llama3.2) (calls the tool, interprets results, responds in natural language)
```

The key point: `log-reader-mcp` always reads from `logs/logs.log` relative to its working directory. No file path needs to be specified in your prompts — it is fixed at the server level.

Prerequisites

Ensure the following are available on your Kali Linux machine before starting:

- Docker (version 28.x or later)
- Docker Compose (version 2.x or later)
- OpenWebUI running in Docker on port 3000
- Your log file at `/home/kali/logs/logs.log` Download the log file from the Github repo and paste it in the mentioned directory (/home/kali/logs/) This prerequisite and file structure is compulsory for the practical to work as the contents for the files to be created after this step are mentioned accordingly.

Verify Docker and Docker Compose are installed:

```
docker --version
docker-compose --version
```

Step 1 — Create the Docker Compose File

The Docker Compose file launches a container that installs and runs both `mcpo` and `log-reader-mcp` together.

Open a terminal and create the file using nano:

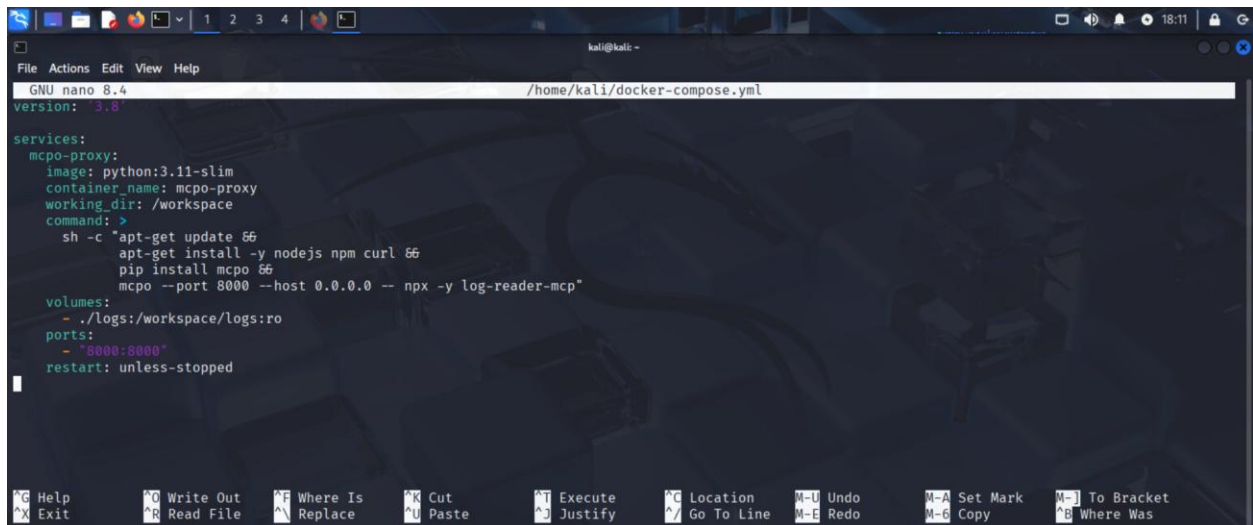
```
nano ~/docker-compose.yml
```

```
(kali㉿kali)-[~]
$ nano ~/docker-compose.yml
```

Type the following content exactly as shown:

```
services:
  mcpo-proxy:
    image: python:3.11-slim
    container_name: mcpo-proxy
    working_dir: /workspace
    command: >
      sh -c "apt-get update &&
        apt-get install -y nodejs npm curl &&
        pip install mcpo &&
        mcpo --port 8000 --host 0.0.0.0 -- npx -y log-reader-mcp"
volumes:
```

```
- ./logs:/workspace/logs:ro
ports:
- "8000:8000"
restart: unless-stopped
```



```
GNU nano 8.4 /home/kali/docker-compose.yml
version: 3.8

services:
  mcpo-proxy:
    image: python:3.11-slim
    container_name: mcpo-proxy
    working_dir: /workspace
    command: >
      sh -c "apt-get update &&
      apt-get install -y nodejs npm curl &&
      pip install mcpo &&
      mcpo --port 8000 --host 0.0.0.0 -- npx -y log-reader-mcp"
    volumes:
      - ./logs:/workspace/logs:ro
    ports:
      - "8000:8000"
    restart: unless-stopped
```

Save the file with **Ctrl+O**, press Enter, then exit with **Ctrl+X**.

Note: The `./logs` volume mount maps your existing `~/logs/` folder on the host into the container at `/workspace/logs`. Since the working directory inside the container is `/workspace`, `log-reader-mcp` will find your log file at `logs/logs.log` automatically.

Step 2 — Start the MCPO Container

Run the following command from your home directory where the `docker-compose.yml` file is located:

```
cd ~
docker-compose up -d
```



```
(kali@kali)~$ docker-compose up -d
```

Docker will pull the `python:3.11-slim` image and start the container. This may take a minute on first run.

Follow the startup logs to confirm MCPO is ready:

```
docker logs -f mcpo-proxy
```

```
(kali㉿kali)-[~]  
$ docker logs -f mcpo-proxy
```

Wait until you see this line:

```
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Press **Ctrl+C** to stop following the logs. The container continues running in the background.

Step 3 — Verify MCPO is Accessible

Confirm that MCPO is listening on port 8000:

```
curl http://localhost:8000/docs
```

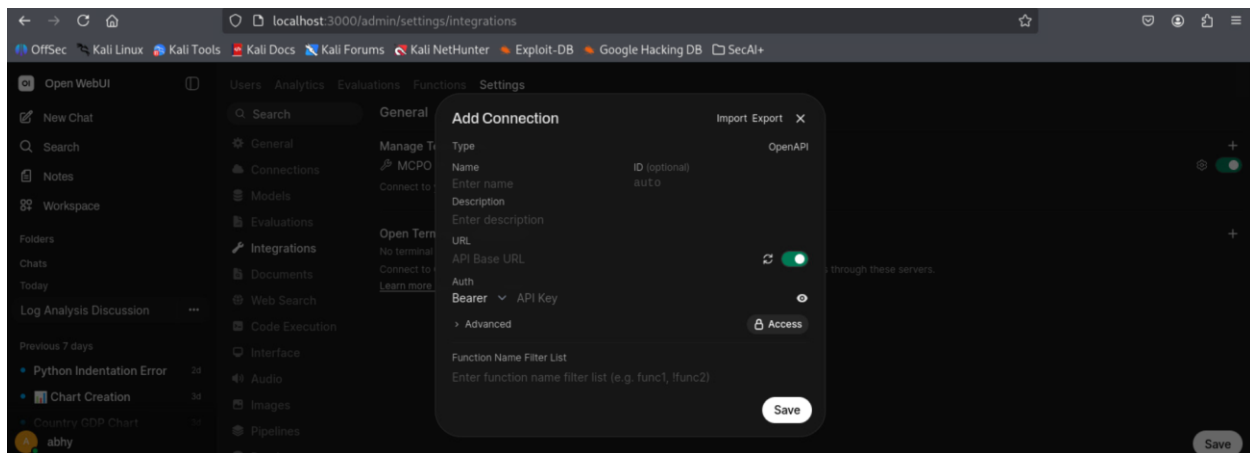
You should receive an HTML response containing the Swagger UI for `log-reader-mcp`. You can also open `http://localhost:8000/docs` in your browser to see the interactive API documentation.

Step 4 — Connect MCPO to OpenWebUI

OpenWebUI runs inside Docker and cannot use `localhost` to reach MCPO on the host. It must use the Docker gateway IP instead, which is `172.17.0.1`.

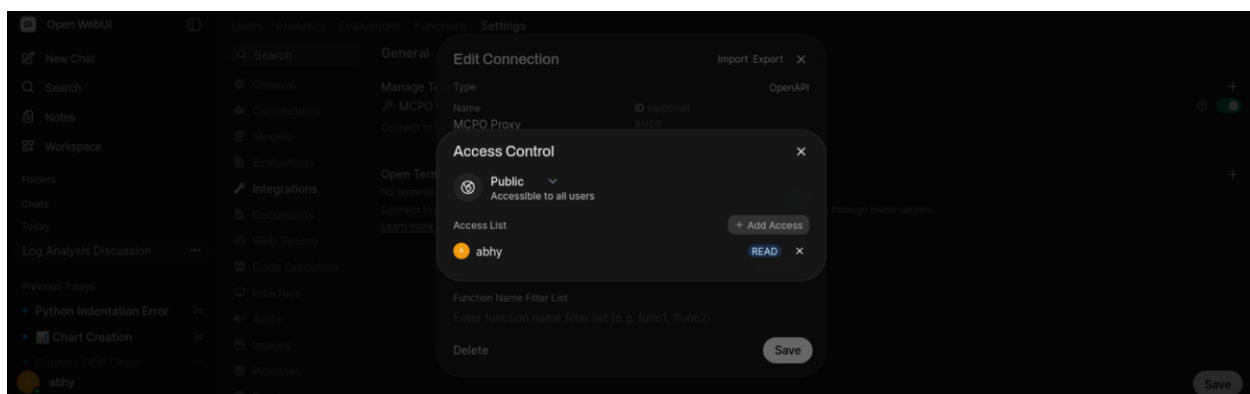
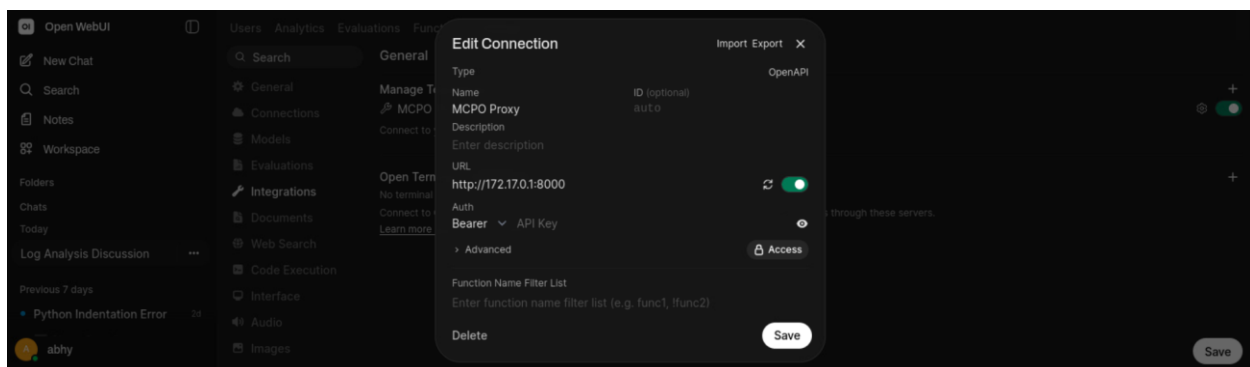
Follow these steps in OpenWebUI:

1. Open OpenWebUI at `http://localhost:3000`
2. Click your profile icon and select **Admin Panel**
3. Navigate to **Settings** → **Integrations**
4. Under **Manage Tool Servers**, click the **+** button



5. Fill in the connection form:

Field	Value
Name	MCPO Proxy
URL	http://172.17.0.1:8000
Visibility	Public (For this click on Access lock icon). Here, click on + Add Access and select your user name.
Auth	Leave blank



6. Click **Save**

Note: The Name field is important — it determines what label appears in the chat tools menu. Setting it to 'MCPO Proxy' ensures it displays correctly for all users.

Now perform Lab 3.2. Refer to walkthrough from the GitHub Repository

Troubleshooting

MCPO Proxy not appearing in chat tools

- Ensure the tool was added under **Admin Settings**, not user settings
- Ensure Visibility is set to **Public**
- Verify the container is running: `docker ps | grep mcpo`

Tool call returns no data

- Confirm the log file exists at `~/logs/logs.log`
- Check container logs: `docker logs mcpo-proxy`

Port 8000 not accessible

- Verify the container started: `docker ps`
- Test connectivity: `curl http://localhost:8000/docs`

Do this only after you are done with Lab 3.2:

Stopping the MCPO Container (Optional)

To stop the container when you are done:

```
cd ~  
docker-compose down
```

To restart it later:

```
cd ~
```

```
docker-compose up -d
```

If you don't stop it, MCPO container will still be up and running if you Restart the system.